

## **Activity 05: Using Semaphores to Prevent Deadlock**

Sharie Rhea

Southern New Hampshire University

CS 510: Operating Systems Principles and Applications

Brian Enochson

March 8, 2026

The provided Python file, `semaphore.py`, demonstrates the utility of semaphores by creating 10 processes to download information from various webpages, but limits concurrent access to only 4 processes at a time. Each process displays a timestamp and its status at the 3 stages of executing the critical section: wait, acquisition, and release. No major challenges were encountered during the creation of this script.

To run the program, please follow the steps in the included `README.md`.

When properly used, semaphores prevent deadlocks by ensuring that a certain number of processes always have access to the desired resource. The initialization value of the semaphore determines the number of processes that may concurrently access the critical section (Silberschatz et al. 2023). Increasing this value will speed up the execution time of the script provided here, as it allows more processes to query their webpages simultaneously. In other applications, this value must be carefully considered, as a value that is too high could overload a resource and result in deteriorating performance instead of improving it. It should be noted that a semaphore with an initial value of 1 is effectively a mutex lock (Silberschatz et al. 2023). A semaphore with an initial value of 0 is invalid, as no processes will ever be able to acquire the semaphore and execute their critical sections.

Deadlock is a state where all executing processes are currently waiting for a condition that can only occur from the execution of other processes (Silberschatz et al. 2023). Four conditions are necessary for deadlock to occur (Silberschatz et al. 2023). First, mutual exclusion must be guaranteed. Each process must also acquire some resource and hold it even while waiting to enter the critical section. Processes must not be preempted and forced to release their resources. Finally, each process must wait for another in a circular fashion such that no processes

will release their resource and break the deadlock. Deadlock prevention is necessary to protect the performance and integrity of a system. Once a deadlock occurs, any operating system resources being used by those processes are essentially lost until manual intervention is performed. In extreme cases, deadlock can cause a system to completely freeze or lock up, requiring a hard reset (Silberschatz et al. 2023).

Most mechanisms for preventing deadlocks rely on ensuring controlled access to a critical section of code. Mutex locks, semaphores, and monitor constructs are all based on this concept. Below their high-level application usage, these programming structures work because there are underlying hardware instructions that are guaranteed to be atomic. These instructions—namely `test_and_set()` and `compare_and_swap()`—control the modification of the lock's value and prevent it from becoming desynchronized (Silberschatz et al. 2023). For semaphores, each call to acquire the semaphore using the `wait()` function decrements the semaphore's value if possible or blocks the process until it can. After a process finishes its critical section it must call `signal()` to increment the semaphore and allow another process to acquire it. The `wait()` and `signal()` functions are implemented using the atomic hardware instructions mentioned. If `wait()` and `signal()` are not called properly in the application code, then the semaphore will not have the desired effect. Debugging and analyzing concurrent applications is difficult, as troublesome behavior may be flaky and only surface occasionally. Often, even when a deadlock is possible, one will not occur unless a specific execution order is chosen (Silberschatz et al. 2023).

## References

Silberschatz, A., Galvin, P. B., & Gagne, G. (2023). *Operating System Concepts* (10th ed.).

Wiley. <https://learn.zybooks.com/zybook/CS-510-10435.202617-1>